

Trabalho 1

Experimentos em Complexidade

Professor: Gabriel Fonseca Silva

Adaptação: Prof. Isabel H. Manssour e Prof. Eduarda R. Monteiro

Elaborado por: Prof. João B. Oliveira

Disciplina: 4645G-04 – Algoritmos e Estruturas de Dados I

Escola Politécnica – PUCRS

Semestre 2026/2

3 de setembro de 2026

Resumo

Este trabalho tem o objetivo de analisar o desempenho de algoritmos usando técnicas experimentais. A análise experimental não substitui uma análise formal de complexidade, mas permite observar e comparar o comportamento dos algoritmos para diferentes tamanhos de entrada. Os algoritmos a serem analisados serão disponibilizados em documentos específicos.

1 Introdução

Em muitos momentos, você vai querer saber se o algoritmo que desenvolveu é eficiente, ou seja, se ele é uma boa maneira de resolver determinado problema. Nem sempre essa pergunta tem uma resposta simples: tudo depende do problema, da solução escolhida, das entradas e de vários outros fatores.

Analisar experimentalmente o comportamento de um algoritmo é uma maneira direta de observar como seu custo cresce. Este trabalho apresenta uma técnica simples para realizar essa análise. Antes de começar, porém, é bom ter em mente duas limitações:

1. Só podemos aplicar a técnica depois que o programa estiver implementado, testado e funcionando corretamente; se o resultado mostrar que o algoritmo não é adequado, uma parte do trabalho já terá sido feita;
2. A análise depende da execução do programa e dos dados escolhidos. Uma amostra muito pequena, muito favorável ou pouco representativa pode levar a conclusões erradas.

Mesmo com essas limitações, a análise experimental nos ajuda a levantar hipóteses, comparar soluções e prever como o número de operações muda à medida que a entrada aumenta.

2 Os fundamentos

A ideia central é simples: executar um algoritmo para vários tamanhos de entrada e observar como seu custo cresce. Vamos usar o Fibonacci recursivo como exemplo:

```
long f( long n )
    if( n <= 1 ) return 1
    return f(n - 1) + f(n - 2)
```

Depois de implementar o algoritmo, queremos entender como ele se comporta quando n varia. Há duas maneiras simples de coletar essas informações.

Medindo tempos

Uma opção é registrar o instante logo antes e logo depois da execução e medir a diferença. Ao repetir o processo para vários valores de n , montamos uma tabela de tempos. Esse método é útil, mas os resultados podem variar por causa do sistema operacional, do aquecimento da máquina virtual Java, de atividades em segundo plano e das características do computador usado.

Medindo operações

Outra opção é escolher uma operação importante e adaptar o programa para contá-la. No Fibonacci recursivo, por exemplo, podemos contar as chamadas da função. Para isso, incrementamos `cont_op` no início de cada chamada:

```
long f( long n )
    cont_op++
    if( n <= 1 ) return 1
    return f(n - 1) + f(n - 2)

main()
    for( i = 0; i <= 30; i += 1 )
        cont_op = 0
        f(i)
        print i, cont_op
```

Os primeiros resultados são 1, 1, 3, 5, 9, 15, 25, 41, 67, 109, Atenção: esses valores não são os números de Fibonacci, mas a quantidade de chamadas feitas para calculá-los. A Figura 1 mostra os dados até $n = 30$.

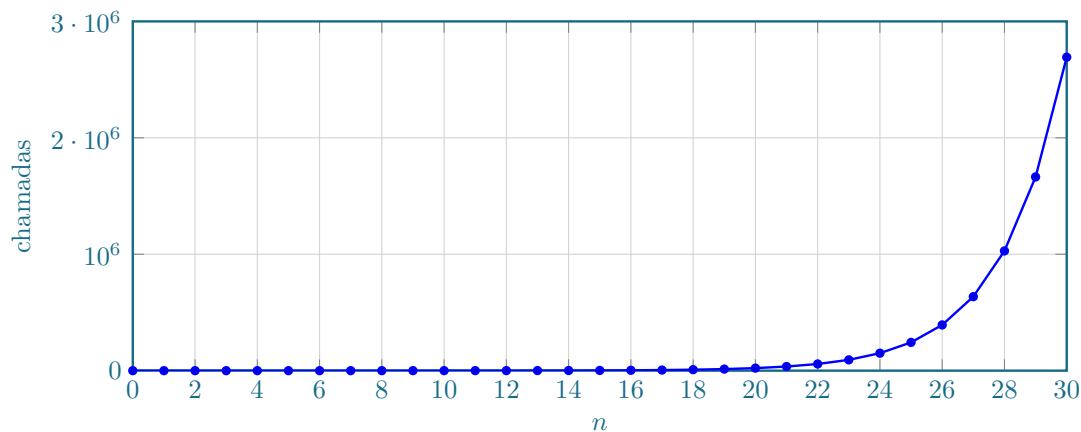


Figura 1: Número de chamadas realizadas pelo Fibonacci recursivo.

Os valores crescem rapidamente, mas apenas esse gráfico não nos diz se o crescimento se parece com n^2 , n^5 , uma exponencial ou outra função. O que buscamos é uma função que mostre o ritmo de crescimento do número de operações e nos ajude a prever o custo para valores que ainda não foram medidos.

Funções polinomiais e exponenciais

Na análise de algoritmos, duas famílias de funções aparecem com bastante frequência.

Funções polinomiais. Nelas, o tamanho do problema aparece como base de potências com expoentes fixos. Por exemplo,

$$f(n) = n^3 + 4n + 16.$$

Até mesmo um polinômio como

$$f(n) = n^{200} + n^{199} + \dots + n + 1$$

continua sendo um polinômio, embora um expoente tão alto esteja longe de ser desejável em um algoritmo. Em geral, quanto menor o maior expoente, mais devagar o custo cresce.

Funções exponenciais. Nelas, o tamanho do problema aparece no expoente, como em

$$f(n) = 3 \cdot 2^n + 11.$$

Quando aumentamos n em uma unidade, o termo principal praticamente dobra. Por isso, um algoritmo exponencial pode ficar inviável mesmo com entradas que não parecem tão grandes. Por exemplo, passar de $n = 5$ para $n = 15$ pode significar aproximadamente mil vezes mais trabalho; passar de $n = 5$ para $n = 25$, cerca de um milhão de vezes mais.

3 Descobrimos a função

Vamos chamar de *função característica* a função que descreve o ritmo de crescimento do número de operações. Para investigar se esse crescimento é exponencial, começamos com

$$f(n) = a \cdot b^n.$$

Não dá para determinar a e b apenas olhando para a curva da Figura 1. Uma transformação logarítmica nos dá uma informação mais útil:

$$\begin{aligned} \log f(n) &= \log(a \cdot b^n) \\ &= \log a + \log(b^n) \\ &= n \log b + \log a. \end{aligned}$$

O lado direito tem a forma de uma reta,

$$\log f(n) = rn + s,$$

em que $r = \log b$ e $s = \log a$. Portanto, se usarmos uma escala logarítmica no eixo vertical e os dados formarem aproximadamente uma reta, teremos um bom indício de crescimento exponencial. É isso que vemos na Figura 2.

Usando os valores $f(2) = 3$ e $f(30) = 2692537$, obtemos uma estimativa da inclinação:

$$r \approx \frac{\log(2692537) - \log(3)}{30 - 2} \approx 0,48955.$$

Como $r = \log b$, segue que

$$b = e^r \approx 1,63158.$$

Para uma estimativa experimental simples, esse resultado é bastante razoável.

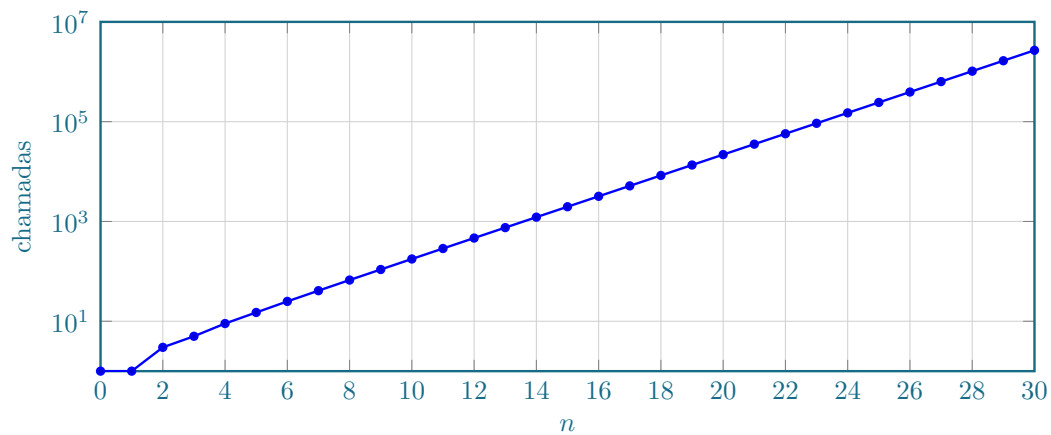


Figura 2: Contagem do Fibonacci com o eixo vertical em escala logarítmica.

4 O caso polinomial

Agora, vamos analisar outro algoritmo:

```
long f( long n )
    long i, j, r = 0
    for( i = 0; i <= n; i += 1 )
        for( j = i + 2; j <= 2 * n; j += 1 )
            r = r + 1
    return r
```

Neste caso, o próprio valor de r já representa a contagem de operações. Os primeiros resultados são:

n	0	1	2	3	4	5	6	7	8	9
$f(n)$	0	1	6	14	25	39	56	76	99	125

A Figura 3 apresenta a curva até $n = 100$.

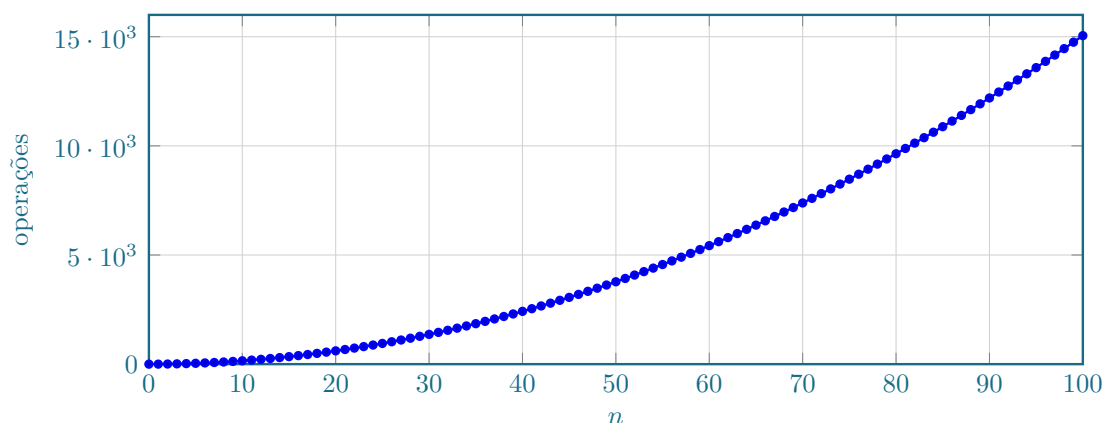


Figura 3: Número de operações do exemplo polinomial.

O crescimento é rápido, mas o gráfico em escala normal ainda não deixa clara a forma da função. Se colocarmos apenas o eixo vertical em escala logarítmica, obteremos a Figura 4. Diferentemente do exemplo exponencial, os pontos não formam uma reta.

Se a função característica for aproximadamente uma potência

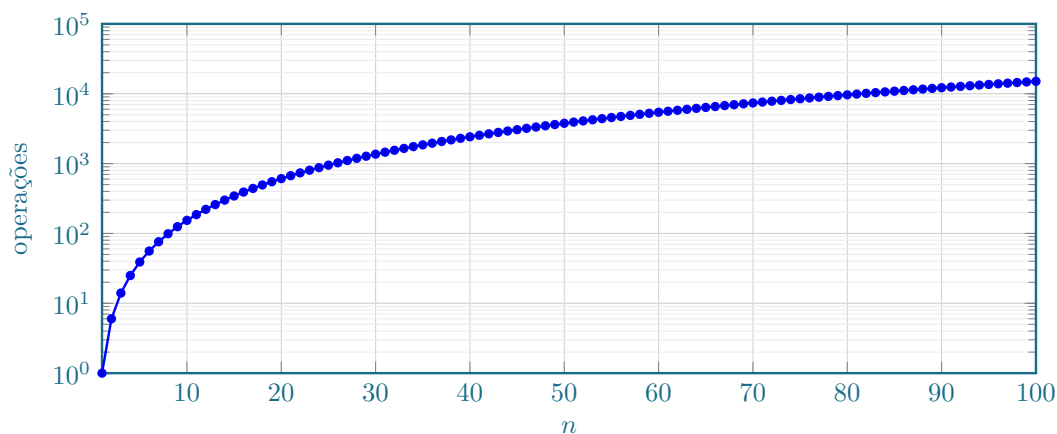


Figura 4: Exemplo polinomial com apenas o eixo vertical logarítmico.

$$f(n) = an^p,$$

então

$$\log f(n) = p \log n + \log a.$$

Agora a reta aparece quando usamos escala logarítmica nos dois eixos. A inclinação dessa reta nos dá uma estimativa do expoente p , como mostra a Figura 5. Neste gráfico, usamos todos os valores de n entre 1 e 10 e, depois, valores de 10 em 10 até chegar a 100.

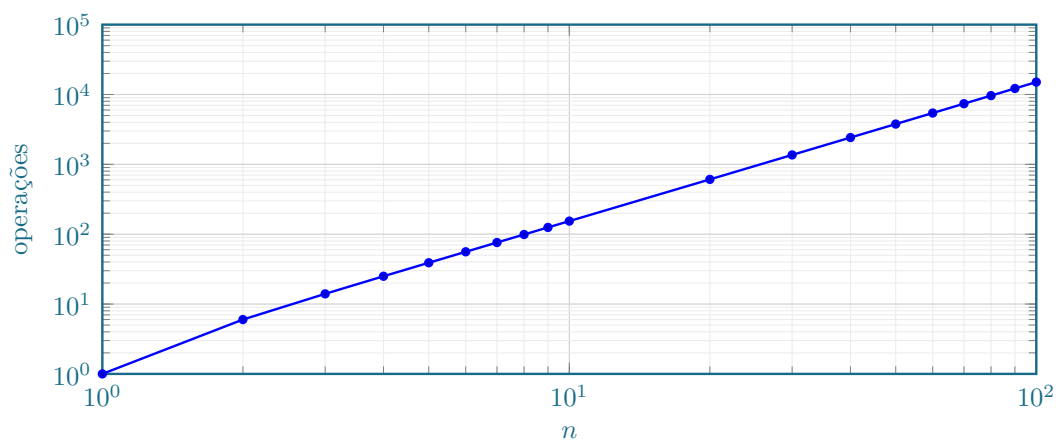


Figura 5: Exemplo polinomial com os dois eixos em escala logarítmica.

Usando $f(1) = 1$ e $f(100) = 15049$, a inclinação estimada é

$$p \approx \frac{\log(15049) - \log(1)}{\log(100) - \log(1)} \approx 2,08875.$$

Isso sugere uma função próxima de $n^{2,09}$. Isto tudo ainda é um resultado experimental e sujeito a algum erro. Por exemplo, se usarmos $f(3) = 14$ e $f(100) = 15049$, chegaremos a $p \approx (1,99)$, o que reforça a hipótese de que o expoente verdadeiro seja 2. O valor experimental não é exatamente dois porque a contagem completa inclui constantes e termos de menor grau, que têm mais influência quando as entradas são pequenas.

Outras famílias

Nem toda função relevante é uma potência pura ou uma exponencial. Também podemos encontrar funções constantes, logarítmicas, proporcionais a $\sqrt{n}$, lineares, $n \log n$ ou polinomiais acompanhadas de fatores logarítmicos. A Tabela 1 mostra todas as classes que podem aparecer nos algoritmos desta atividade. A função encontrada experimentalmente pode ter constantes e termos de menor ordem, mas seu termo dominante deverá se aproximar de um dos modelos da tabela.

Classe	Função característica	Ao dobrar n
Constante	1	praticamente não muda
Logarítmica	$\log n$	aumenta por uma constante
Raiz quadrada	$\sqrt{n}$	multiplica por $\sqrt{2}$
Linear	n	multiplica por 2
Linear-logarítmica	$n \log n$	multiplica por pouco mais de 2
Quadrática	n^2	multiplica por 4
Quadrática-logarítmica	$n^2 \log n$	multiplica por pouco mais de 4
Cúbica	n^3	multiplica por 8
Grau 4	n^4	multiplica por 16
Grau 5	n^5	multiplica por 32

Tabela 1: Classes de complexidade que podem aparecer na atividade.

Se duas hipóteses parecerem igualmente possíveis, uma boa estratégia é normalizar os dados. Essa estratégia pode ser aplicada a qualquer função candidata $g(n)$: se $f(n)/g(n)$ tende a se estabilizar em uma constante positiva para valores crescentes de n , isso fornece evidência de que $g(n)$ descreve adequadamente o crescimento. Por exemplo, para testar $f(n) = an \log n$, observe

$$\frac{f(n)}{n \log n}.$$

Se essa razão começar a se estabilizar conforme n cresce, teremos um bom indício a favor do modelo. Em qualquer caso, use vários pontos e compare mais de uma hipótese.

5 O software

Você pode construir os gráficos com ferramentas como gnuplot, planilhas eletrônicas ou bibliotecas de programação. No gnuplot, por exemplo, considerando um arquivo `dados.txt` com duas colunas, estes são alguns comandos básicos:

```
set grid
plot "dados.txt" with linespoints

set logscale y
plot "dados.txt" with linespoints

set logscale xy
plot "dados.txt" with linespoints
```

Qualquer que seja a ferramenta escolhida, guarde os dados brutos e identifique claramente os eixos, as unidades, as escalas e os valores usados.

6 A missão

Para cada algoritmo recebido, façam o seguinte:

1. Implementar o pseudocódigo em Java sem alterar seu comportamento;
2. Testar a implementação com valores pequenos de n ;
3. Executar o programa com uma sequência adequada de entradas;
4. Registrar n e `cont_op` em um arquivo TXT ou CSV;
5. Criar um gráfico em escala normal;
6. Criar pelo menos um gráfico transformado que ajude a testar a hipótese;
7. Comparar pelo menos duas funções candidatas. A comparação pode ser feita por meio dos gráficos transformados, pela normalização $f(n)/g(n)$, ou por outra análise que mostre qual modelo se ajusta melhor aos dados.
8. Apontar a função característica que melhor explica os dados;
9. Justificar a conclusão com base nos dados, nos gráficos e na estrutura dos laços.

O foco é descobrir como o número de operações de cada algoritmo cresce. Vocês não precisam explicar qual problema prático o algoritmo resolveria. Nesta etapa, apresentem explicitamente a função característica: indicar apenas uma classificação com as notações $O(\cdot)$, $\Omega(\cdot)$ ou $\Theta(\cdot)$ não substitui a análise.

7 Convenções dos algoritmos

Os algoritmos usam um pseudocódigo parecido com C e Java. Para interpretá-lo, sigam estas regras:

- n é um número inteiro e $n \geq 2$;
- A indentação determina o corpo de cada comando;
- A divisão entre variáveis inteiras é uma divisão inteira;
- Os limites dos laços são inclusivos quando usam `<=` ou `>=`;
- A expressão de atualização ocorre depois da execução do corpo;
- Somente o incremento indicado de `cont_op` deve ser contado;
- A implementação Java deve usar `long` para índices, resultados auxiliares e contadores;
- Evitem entradas que causem estouro numérico ou levem a uma execução longa demais e expliquem essa escolha no relatório.

8 Entregáveis

A entrega deve incluir:

1. Um relatório em PDF com metodologia, tabelas, gráficos, análise e conclusões;
2. Os arquivos-fonte Java, um para cada algoritmo;
3. Os arquivos TXT ou CSV utilizados para produzir os gráficos.

Identifiquem os autores e a turma em todos os arquivos. Também deve ser possível reproduzir os dados e os gráficos do relatório usando os códigos-fonte e os arquivos entregues.

9 Critérios de avaliação

Critério	Peso
Implementação e instrumentação corretas	20%
Qualidade e reprodutibilidade dos dados	20%
Construção e apresentação dos gráficos	20%
Identificação e justificativa das funções	25%
Conclusões e qualidade do relatório	15%

Resultados sem os dados correspondentes ou sem uma implementação que possa ser reproduzida podem perder pontos. Da mesma forma, apresentar apenas a função final, sem explicar por que ela foi escolhida, não é uma análise completa.

Checklist antes de entregar

Antes de enviar os arquivos, confirmem se:

- Cada algoritmo tem sua implementação Java correspondente;
- Todas as tabelas mostram claramente os valores de n e `cont_op`;
- Os eixos e as escalas de todos os gráficos estão identificados;
- Cada função proposta está apoiada por dados e por uma justificativa;
- O PDF, os fontes Java e os arquivos TXT ou CSV estão corretamente identificados.